

Alunos: Pedro Henrique Gimenez - 23102766 e Victória Rodrigues Veloso - 23100460

1. Exercício 1

Para o primeiro exercício, o um programa que calcula o fatorial de um número foi desenvolvido. Ele recebe o valor do número à ser calculado via teclado, realiza o cálculo sem procedimentos e exibe o resultado no console.

1.1 Implementação

O código é organizado em duas seções principais: `.data` e `.text`. Na seção `.data`, são definidas variáveis globais para armazenar o resultado do fatorial (`resultado`) e o número fornecido pelo usuário (`n`). Além disso, há strings para imprimir mensagens no console. Na seção `.text`, encontram-se as instruções executáveis do programa.

O programa segue os seguintes passos durante sua execução: solicitação do valor de `n`, pelo usuário, leitura do valor de `n` fornecido pelo usuário, cálculo do fatorial de `n` usando um loop, armazenamento do resultado do fatorial na variável `resultado`, impressão da mensagem indicando o valor do fatorial, impressão do resultado do fatorial.

O cálculo do fatorial é feito através de um loop que decrementa 1 do valor de '`n`' e é executado até que o valor de '`n`' seja igual à uma variável de controle `$t1`.

```
fatorial:
    li $t1, 1 #controla o loop
    li $t2, 1 #armazena as multiplicações sucessivas

    loop:

        mul $t2, $t0, $t2 #(n-1)*n
        subi $t0, $t0, 1 #n-1
        bne $t0, $t1, loop #procura se n = 1

        sw $t2, 0($s0) #armazena o resultado na memória

# ---- imprime nova linha ----
li $v0, 4
la $a0, valorResultado
syscall

# ---- imprime resultado ----
li $v0, 1
li $a0, 0
add $a0, $a0, $t2
syscall
```

1.2 Execução do programa

Para exemplificar a execução do programa, vamos simular o cálculo do fatorial com $n = 5$.

```
Insira o valor de n: 5
```

Figura 1. Leitura do valor de entrada no teclado

\$t0	8	4
\$t1	9	1
\$t2	10	5

Figura 2. Valores dos registradores na primeira iteração do loop

Após à primeira iteração, temos o decremento de 'n' que está no registrador \$t0, e o armazenamento das multiplicações acumuladas em \$t2

\$t0	8	1
\$t1	9	1
\$t2	10	120

Figura 3. Valores dos registradores na última iteração do loop

Na última iteração, temos o valor de 'n' igual à 1 e o valor do resultado do fatorial igual à 120, com isso, o resultado é armazenado na memória, conforme figuras 4 e 5.

\$s0	16	268500992
------	----	-----------

Figura 4. Registrador que armazena o endereço base em que o resultado será armazenado

Address	Value (+0)
268500992	120

Figura 5. Resultado do fatorial armazenado na memória

```
Insira o valor de n: 5
O fatorial de n é: 120
```

Figura 6. Exibição do resultado no console

2. Exercício 2

Para o exercício 2, a implementação do cálculo fatorial também foi implementado. Contudo, para esse script, fizemos o uso do procedimento recursivo.

2.1 Implementação

Assim como no exercício anterior, o programa solicita o valor de n pelo usuário, realiza a leitura e cálculo do fatorial e exibe o resultado no console. No trecho de código abaixo é possível analisar o procedimento recursivo que faz o cálculo do fatorial. Nele, enquanto o valor de "n" não for igual à zero, subtraímos 1 de "n" e adicionamos na pilha

```

fatorial:
    addi $sp, $sp, -8          # incrementa a pilha para armazenar RA e
valor
    sw    $ra, 0($sp)         # salva o ra atual no 1o slot
    sw    $s0, 4($sp)         # salva o n atual no 2o slot

    li    $v0, 1              # retorna 1
    beq   $a0, $zero, return   # se caso, n chegar em 0, a funcao acaba

    add    $s0, $a0, $zero
    addi   $a0, $a0, -1        # subtrai 1 de n
    jal    fatorial

    mul    $v0, $s0, $v0       # multiplicação recursiva

return:
    lw     $ra, 0($sp) #recupera o endereço de retorno na pilha
    lw     $s0, 4($sp) #recupera o endereço de n-1 na pilha
    addi   $sp, $sp, 8 #limpa a pilha
    jr     $ra               #volta para o último endereço recuperado

```

2.2 Execução do programa

Para a execução do exercício 2, também iremos simular o cálculo do fatorial utilizando o número 5 como entrada.

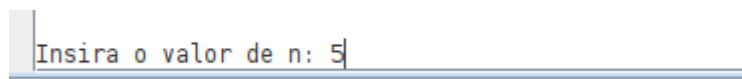


Figura 7. Entrada do usuário no teclado



Figura 8. Argumento que será passado para o procedimento

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
2147479488	0	0	0	4194408	1	4194408	2	4194408
2147479520	3	4194408	4	4194408	5	4194336	0	0

Figura 9. Armazenamento dos dados na pilha

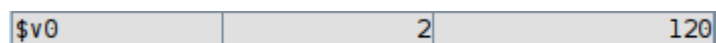


Figura 10. Armazenamento do resultado no registrador de retorno

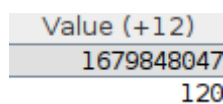


Figura 11. Resultado armazenado na memória

```
Insira o valor de n: 5
O fatorial de n é: 120
```

Figura 11. Impressão do resultado no console

3. Exercício 3

3.1 Simulações BHT Fatorial Sem Procedimentos

Para o primeiro fatorial, comparamos se o valor de “n”, na instrução de desvio condicional, é igual a 1. Caso negativo, ele realiza o salto, caso positivo o valor é armazenado na memória. Como pode ser visto no seguinte código:

loop:

```
mul $t2, $t0, $t2    # fat = (n-1)*n
subi $t0, $t0, 1     # n -= 1
bne $t0, $t1, loop   # se n != 1, volta pro loop

sw $t2, 0($s0)
```

Para todos os testes deste código iremos utilizar **BHT Entries** igual a 8, pois só há uma instrução condicional no fatorial sem procedimentos. O **valor inicial de n** em todos os testes será 4.

3.1.2 Simulação 01

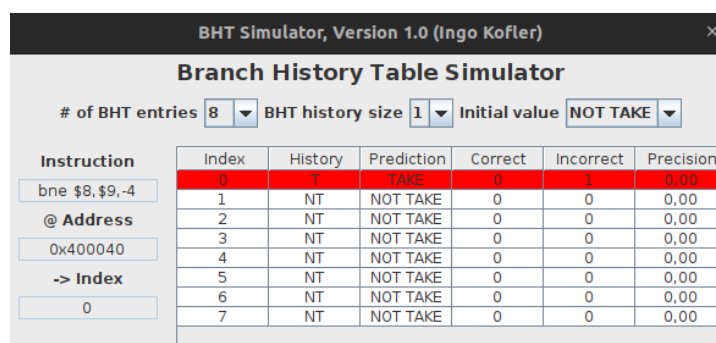
A primeira simulação foi realizada utilizando os seguintes parâmetros no *BHT Simulator*:

BHT Entries: 8

BHT Size: 1 bit

Initial Value: NOT TAKE

Ao iniciar a simulação com *NOT TAKE*, irá acontecer uma falha (Figura 12) pois n está em $n-1 = 3$ (que é diferente de 1) e o branch será *TAKEN*, pois o loop precisa ser repetido, ou seja, o salto é efetuado. Isso faz com que o simulador mude a próxima predição para *TAKE*.



BHT Simulator, Version 1.0 (Ingo Kofler)

Branch History Table Simulator

of BHT entries: 8 BHT history size: 1 Initial value: NOT TAKE

Instruction	Index	History	Prediction	Correct	Incorrect	Precision
bne \$8,\$9,-4	0	1	TAKE	0	1	0,00
@ Address	1	NT	NOT TAKE	0	0	0,00
0x400040	2	NT	NOT TAKE	0	0	0,00
-> Index	3	NT	NOT TAKE	0	0	0,00
0	4	NT	NOT TAKE	0	0	0,00
	5	NT	NOT TAKE	0	0	0,00
	6	NT	NOT TAKE	0	0	0,00
	7	NT	NOT TAKE	0	0	0,00

Figura 12. Falha no primeiro chute (NOT TAKE) e mudança para take

Após a mudança de *NOT TAKE* para *TAKE*, o simulador acertou (Figura 13) pois *n* está em 2 que é diferente de 1 (Figura 14) e o branch foi realizado. A precisão se tornou 50% pois errou o primeiro e acertou o segundo.

Index	History	Prediction	Correct	Incorrect	Precision
0	T	TAKE	1	1	50,00
1	NT	NOT TAKE	0	0	0,00
2	NT	NOT TAKE	0	0	0,00
3	NT	NOT TAKE	0	0	0,00
4	NT	NOT TAKE	0	0	0,00
5	NT	NOT TAKE	0	0	0,00
6	NT	NOT TAKE	0	0	0,00
7	NT	NOT TAKE	0	0	0,00

Figura 13. Acerto após mudança

\$t0	8	2
------	---	---

Figura 14. Registrador de *n* está em 2

E, por fim, após realizar todas as iterações de *n* até chegar em *n* igual a 1, o simulador irá errar (Figura 15) no último caso pois ele estava como *TAKE* devido as iterações anteriores. Porém como *n* está em 1, não irá ocorrer o branch no programa. Totalizando 33% de precisão com 2 erros (sempre será 2: o inicial e o final) e 1 acerto (que será exatamente *n*-3).

Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	1	2	33,33
1	NT	NOT TAKE	0	0	0,00
2	NT	NOT TAKE	0	0	0,00
3	NT	NOT TAKE	0	0	0,00
4	NT	NOT TAKE	0	0	0,00
5	NT	NOT TAKE	0	0	0,00
6	NT	NOT TAKE	0	0	0,00
7	NT	NOT TAKE	0	0	0,00

Figura 15. Simulador errou o último caso (*n*=1)

Analisando o resultado, temos que o simulador com 1 bit e errando o chute inicial sempre errará 2 vezes (inicial e o final) e acertará *n*-3 vezes.

3.1.3 Simulação 02

BHT Entries: 8

BHT Size: 1 bit

Initial Value: TAKE

Para a segunda simulação, iniciamos com *TAKE*, ou seja, consideramos que o desvio é tomado (e realmente será). Com isso, a simulação obtém 100% de precisão no primeiro chute. Na figura 16 podemos observar.

Index	History	Prediction	Correct	Incorrect	Precision
0	T	TAKE	1	0	100,00
1	T	TAKE	0	0	0,00
2	T	TAKE	0	0	0,00
3	T	TAKE	0	0	0,00
4	T	TAKE	0	0	0,00
5	T	TAKE	0	0	0,00
6	T	TAKE	0	0	0,00
7	T	TAKE	0	0	0,00

Figura 16. Acerto inicial tomando desvio tomado TAKE para $n-1 = 3$

\$t0	8	3
\$t1	9	1

Figura 17. Valores dos registradores no momento da comparação no desvio condicional (\$t0 = n e \$t1 é a constante comparadora = 1)

Já na última iteração do loop, quando \$t0 armazena um valor igual a \$t1, conforme figura x+6, o desvio não será tomado e o valor será armazenado na memória. Logo, o simulador erra, como pode ser visto na Figura 19.

\$t0	8	1
\$t1	9	1

Figura 18. Valores dos registradores no momento da comparação no desvio condicional

Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	3	1	75,00
1	T	TAKE	0	0	0,00
2	T	TAKE	0	0	0,00
3	T	TAKE	0	0	0,00
4	T	TAKE	0	0	0,00
5	T	TAKE	0	0	0,00
6	T	TAKE	0	0	0,00
7	T	TAKE	0	0	0,00

Figura 19. Simulador com erro na última iteração do loop

```

Insira o valor de n: 4
O fatorial de n é: 24

```

Figura 20. Impressão do resultado no console

Assim, pode-se verificar que o simulador neste caso, para qualquer valor de n, sempre errará 1 vez (último caso $n=1$) e acertará as outras $n-1$ iterações.

3.1.4 Simulação 03

BHT Entries: 8

BHT Size: 2 bits

Initial Value: NOT TAKE

Para a simulação 03, utilizamos o BHT de 2 bits iniciado com NOT TAKE e, conforme figura x+9 obtivemos um erro, pois o desvio seria tomado ($n = 3$). Contudo, o simulador de 2 bits só muda a predição quando os dois últimos testes derem erro.

Index	History	Prediction	Correct	Incorrect	Precision
0	NT, T	NOT TAKE	0	1	0,00
1	NT, NT	NOT TAKE	0	0	0,00
2	NT, NT	NOT TAKE	0	0	0,00
3	NT, NT	NOT TAKE	0	0	0,00
4	NT, NT	NOT TAKE	0	0	0,00
5	NT, NT	NOT TAKE	0	0	0,00
6	NT, NT	NOT TAKE	0	0	0,00
7	NT, NT	NOT TAKE	0	0	0,00

Figura 21. Simulador com erro na primeira iteração do loop

Em seguida, após a 2a iteração (n=2), o simulador erra novamente e altera a próxima previsão para TAKE (Figura 22).

Index	History	Prediction	Correct	Incorrect	Precision
0	T, T	TAKE	0	2	0,00
1	NT, NT	NOT TAKE	0	0	0,00
2	NT, NT	NOT TAKE	0	0	0,00
3	NT, NT	NOT TAKE	0	0	0,00
4	NT, NT	NOT TAKE	0	0	0,00
5	NT, NT	NOT TAKE	0	0	0,00
6	NT, NT	NOT TAKE	0	0	0,00
7	NT, NT	NOT TAKE	0	0	0,00

Figura 22. Simulador erra e muda previsão para TAKE

Para a última iteração, o desvio não é tomado (o resultado é armazenado na memória), porém como nossa previsão está em TAKE, erramos novamente e encerramos a simulação com 0% de precisão.

Index	History	Prediction	Correct	Incorrect	Precision
0	T, NT	TAKE	0	3	0,00
1	NT, NT	NOT TAKE	0	0	0,00
2	NT, NT	NOT TAKE	0	0	0,00
3	NT, NT	NOT TAKE	0	0	0,00
4	NT, NT	NOT TAKE	0	0	0,00
5	NT, NT	NOT TAKE	0	0	0,00
6	NT, NT	NOT TAKE	0	0	0,00
7	NT, NT	NOT TAKE	0	0	0,00

Figura 23. Simulador com erro na última iteração do loop

Verifica-se assim que o simulador com 2 bits e a previsão inicial errada sempre vai errar 3 vezes (2 iniciais e a final) e acertar n-4 vezes para $n \geq 4$. Para $n < 4$, ele errará n-2 vezes e acertará 1 vez.

3.1.5 Simulação 04

BHT Entries: 8

BHT Size: 2 bits

Initial Value: TAKE

Para a última simulação do exercício 1, iniciamos a previsão com TAKE e obtivemos acerto Figura 24.

Index	History	Prediction	Correct	Incorrect	Precision
0	T, T	TAKE	1	0	100,00
1	T, T	TAKE	0	0	0,00
2	T, T	TAKE	0	0	0,00
3	T, T	TAKE	0	0	0,00
4	T, T	TAKE	0	0	0,00
5	T, T	TAKE	0	0	0,00
6	T, T	TAKE	0	0	0,00
7	T, T	TAKE	0	0	0,00

Figura 24. Simulador com erro na última iteração do loop

Como todos os desvios seguintes também serão tomados, tivemos um erro de predição apenas na última iteração, em que o desvio não é tomado figura x+12.

Index	History	Prediction	Correct	Incorrect	Precision
0	T, NT	TAKE	2	1	66,67
1	T, T	TAKE	0	0	0,00
2	T, T	TAKE	0	0	0,00
3	T, T	TAKE	0	0	0,00
4	T, T	TAKE	0	0	0,00
5	T, T	TAKE	0	0	0,00
6	T, T	TAKE	0	0	0,00
7	T, T	TAKE	0	0	0,00

Figura 25. Simulador com erro na última iteração do loop

3.1.6 Conclusão

Após realizar todas as simulações BHT possíveis no fatorial sem procedimentos utilizando $n=4$, chegamos na seguinte tabela:

Tabela 1. Valores de precisão para os métodos de predição executados

Método de previsão	Take	Not take
BHT 1 bit	75%	33,33%
BHT 2 bits	66,67%	0%

Analisando o desempenho dos métodos de previsão para o script desenvolvido em assembly, o melhor método de predição obtido foi o de **Branch History Table de 1 bit**, quando iniciamos a predição com acerto para o desvio (neste caso, **TAKE**).

3.2 Simulações BHT Fatorial Com Procedimentos

Para o segundo fatorial, são realizadas chamadas de um procedimento recursivo através de jumps. Como um jump não é condicional, ele não será avaliado na tabela BHT.

O único branch do código está dentro do procedimento fatorial que compara se n está igual a 0. Caso positivo, ele realiza o salto para o retorno, caso negativo ele realiza a multiplicação recursiva chamando por jal o próximo $n-1$.

Agora, ao contrário do fatorial anterior, a predição correta é NOT TAKE, pois utilizamos *branch on equal* ao invés de *branch not equal* como era no anterior.

Como pode ser visto no seguinte código:


```

# ===== FUNÇÃO FATORIAL =====
# recebe $a0 (N)
# retorna $v0 (N!)

fatorial:
    addi $sp, $sp, -8          # incrementa a pilha para armazenar RA e
    valor
    sw    $ra, 0($sp)          # salva o ra atual no 1o slot
    sw    $s0, 4($sp)          # salva o n atual no 2o slot

    li    $v0, 1               # retorna 1
    beq   $a0, $zero, return   # se caso, n chegar em 0, a funcao acaba

    add   $s0, $a0, $zero
    addi  $a0, $a0, -1          # subtrai 1 de n
    jal   fatorial

    mul   $v0, $s0, $v0         # multiplicação recursiva

return:
    lw    $ra, 0($sp)
    lw    $s0, 4($sp)
    addi  $sp, $sp, 8
    jr    $ra

```

3.2.1 Simulação 01

BHT Entries: 8

BHT Size: 1 bit

Initial Value: NOT TAKE

Começamos a simulação com NOT TAKE que é o correto (Figura 26), já que ele só pula quando n é igual a 0.

Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	0	0	0,00
1	NT	NOT TAKE	0	0	0,00
2	NT	NOT TAKE	0	0	0,00
3	NT	NOT TAKE	0	0	0,00
4	NT	NOT TAKE	0	0	0,00
5	NT	NOT TAKE	0	0	0,00
6	NT	NOT TAKE	1	0	100,00
7	NT	NOT TAKE	0	0	0,00

Figura 26. Primeira predição correta da simulação com n = 4

Após 4 iterações, ao chegar em n igual a 0, o simulador irá errar a previsão pois dessa vez deveria acontecer o desvio e ele irá alterar a próxima previsão para TAKE (Figura 27). Assim concluindo o programa com 80% de precisão.

Index	History	Prediction	Correct	Incorrect	Precision
0	NT	NOT TAKE	0	0	0,00
1	NT	NOT TAKE	0	0	0,00
2	NT	NOT TAKE	0	0	0,00
3	NT	NOT TAKE	0	0	0,00
4	NT	NOT TAKE	0	0	0,00
5	NT	NOT TAKE	0	0	0,00
6	T	TAKE	4	1	80,00
7	NT	NOT TAKE	0	0	0,00

Figura 27. Simulador erra última iteração com n=0 e altera para TAKE

Concluindo, é possível perceber que o resultado é bem similar ao do 1º fatorial com previsão inicial correta e 1 bit. Em que o programa sempre acertará n vezes e errará 1 vez.

3.2.2 Simulação 02

BHT Entries: 8

BHT Size: 1 bit

Initial Value: TAKE

Começamos a simulação desta vez com TAKE que é o incorreto. Então ele irá errar a primeira previsão e mudar para NOT TAKE (Figura 28).

Index	History	Prediction	Correct	Incorrect	Precision
0	T	TAKE	0	0	0,00
1	T	TAKE	0	0	0,00
2	T	TAKE	0	0	0,00
3	T	TAKE	0	0	0,00
4	T	TAKE	0	0	0,00
5	T	TAKE	0	0	0,00
6	NT	NOT TAKE	0	1	0,00
7	T	TAKE	0	0	0,00

Figura 28. O simulador errou a primeira previsão e alterou para NOT TAKE

Depois, ele irá sempre acertar até chegar em n igual a 0. Quando n chegar a este valor, ele irá errar a previsão e alterar a previsão para TAKE (Figura 29). Totalizando em 3 acertos e 2 erros.

Index	History	Prediction	Correct	Incorrect	Precision
0	T	TAKE	0	0	0,00
1	T	TAKE	0	0	0,00
2	T	TAKE	0	0	0,00
3	T	TAKE	0	0	0,00
4	T	TAKE	0	0	0,00
5	T	TAKE	0	0	0,00
6	T	TAKE	3	2	60,00
7	T	TAKE	0	0	0,00

Figura 29. Acertou todas as próximas iterações menos a última

É possível analisar que essa simulação sempre errará 2 vezes (inicial e final) e acertará $n-1$ vezes.

3.2.3 Simulação 03

BHT Entries: 8

BHT Size: 2 bits

Initial Value: TAKE

Na simulação com 2 bits iniciando com TAKE, ele irá errar as duas primeiras vezes (com $n=4$ e $n=3$) para conseguir mudar para o valor correto (Figura 30).

Index	History	Prediction	Correct	Incorrect	Precision
0	T, T	TAKE	0	0	0,00
1	T, T	TAKE	0	0	0,00
2	T, T	TAKE	0	0	0,00
3	T, T	TAKE	0	0	0,00
4	T, T	TAKE	0	0	0,00
5	T, T	TAKE	0	0	0,00
6	NT, NT	NOT TAKE	0	2	0,00
7	T, T	TAKE	0	0	0,00

Figura 30. Errou as duas primeiras iterações e alterou para NOT TAKE

Agora, com a predição correta, ele irá continuar acertando (Figura 31) até chegar em $n = 0$, em que ele irá errar uma vez e o programa se encerrará (Figura 32).

Index	History	Prediction	Correct	Incorrect	Precision
0	T, T	TAKE	0	0	0,00
1	T, T	TAKE	0	0	0,00
2	T, T	TAKE	0	0	0,00
3	T, T	TAKE	0	0	0,00
4	T, T	TAKE	0	0	0,00
5	T, T	TAKE	0	0	0,00
6	NT, NT	NOT TAKE	2	2	50,00
7	T, T	TAKE	0	0	0,00

Figura 31. O simulador com NOT TAKE acerta enquanto n não chega em 0

Index	History	Prediction	Correct	Incorrect	Precision
0	T, T	TAKE	0	0	0,00
1	T, T	TAKE	0	0	0,00
2	T, T	TAKE	0	0	0,00
3	T, T	TAKE	0	0	0,00
4	T, T	TAKE	0	0	0,00
5	T, T	TAKE	0	0	0,00
6	NT, T	NOT TAKE	2	3	40,00
7	T, T	TAKE	0	0	0,00

Figura 32. Erra a última predição em que $n = 0$

Após o resultado, percebe-se que o simulador, para $n \geq 2$, irá sempre errar 3 vezes (2 casos iniciais e caso final) e acertar $n-2$ vezes.

3.2.4 Simulação 04

BHT Entries: 8

BHT Size: 2 bits

Initial Value: NOT TAKE

O simulador inicia acertando todos os casos, então não mudará para TAKE (Figura 33). E só errará o último caso um pouco antes do programa se encerrar (Figura 34).

Index	History	Prediction	Correct	Incorrect	Precision
0	NT, NT	NOT TAKE	0	0	0,00
1	NT, NT	NOT TAKE	0	0	0,00
2	NT, NT	NOT TAKE	0	0	0,00
3	NT, NT	NOT TAKE	0	0	0,00
4	NT, NT	NOT TAKE	0	0	0,00
5	NT, NT	NOT TAKE	0	0	0,00
6	NT, NT	NOT TAKE	1	0	100,00
7	NT, NT	NOT TAKE	0	0	0,00

Figura 33. Simulador acerta o primeiro caso

Index	History	Prediction	Correct	Incorrect	Precision
0	NT, NT	NOT TAKE	0	0	0,00
1	NT, NT	NOT TAKE	0	0	0,00
2	NT, NT	NOT TAKE	0	0	0,00
3	NT, NT	NOT TAKE	0	0	0,00
4	NT, NT	NOT TAKE	0	0	0,00
5	NT, NT	NOT TAKE	0	0	0,00
6	NT, T	NOT TAKE	4	1	80,00
7	NT, NT	NOT TAKE	0	0	0,00

Figura 34. Simulador erra apenas o último caso

Podemos concluir que o caso de 2 bits iniciando com o valor correto é idêntico ao simulador com 1 bit e caso certo, em que se erra apenas uma vez e acerta n vezes, para qualquer $n \geq 1$.

3.1.6 Conclusão

Após realizar todas as simulações BHT possíveis no fatorial com procedimentos utilizando $n=4$, chegamos na seguinte tabela:

Tabela 1. Valores de precisão para os métodos de predição executados

Método de previsão	Take	Not take
BHT 1 bit	60%	80%
BHT 2 bits	40%	80%

Analisando o desempenho dos métodos de previsão para o script desenvolvido em assembly, os melhores métodos de predição obtidos foram os de **Branch History Table de 1 bit e 2 bits**, quando iniciamos a predição com acerto para o desvio (neste caso, **NOT TAKE**).